SymPy: symbolic differentiation

Setup: symbols are independent variables; `**` means a power.

```
import sympy as sp
x, y = sp.symbols("x y")
f = 3*x**2*y + 2*x - 3*y
```

Partials: `sp.diff(f, x)` differentiates in x, holding y fixed. `sp.diff(f, x, 2)` computes f_{xx}; `sp.diff(f, x, y)` gives $f_{xy} = (f_x)_y$ (first x, then y).

Derivative arrays: the variable order fixes the column order.

```
grad_f = sp.Matrix([sp.diff(f, x), sp.diff(f, y)])
H = sp.hessian(f, (x, y))
Jf = sp.Matrix([f]).jacobian([x, y])
```

Shapes: gradient `(2, 1)`, Hessian `(2, 2)`, scalar Jacobian `(1, 2)`. Here `Jf == grad_f.T` and `grad_f.jacobian([x, y]) == H`. For m component expressions in n variables, `F.jacobian(vars)` has shape `(m, n)`; `vars` lists the inputs. Rows index outputs; columns index inputs. Indices start at zero: `H[0, 1]` is the first-row, second-column entry.

Evaluation and local prediction

Substitute after differentiating: `sp.diff(f, x).subs({x: 1, y: 2})` evaluates $f_x(1, 2)$; substituting first loses the variable dependence. `sp.Rational(1, 10)` keeps $1/10$ exact.

Linear approximation: for column matrices `a`, `h`, and a vector expression `F`,

```
point = {x: a[0], y: a[1]}
Ja = F.jacobian([x, y]).subs(point)
prediction = F.subs(point) + Ja*h
```

This approximates $F(\mathbf{a} + \mathbf{h})$. SymPy uses `*` for matrix products. `Ja.rref()` returns the RREF and pivot indices; `Ja.nullspace()` returns a list of basis columns, of length equal to the nullity. A null-space direction causes zero *first-order* change, not necessarily zero actual change. Use `basis[0]` only for a nonempty basis list.

Chain rule: evaluate, then multiply

Inner map first: if `G` depends on `u, v` and outputs two coordinates, use

```
JG = G.jacobian([u, v])
Jf_at_G = Jf.subs({x: G[0], y: G[1]})
chain = sp.simplify(Jf_at_G * JG)
```

This is $J_{f\circ G} = (J_f \circ G)J_G$: evaluate the outer derivative at the inner output. Compare with a direct derivative using `sp.simplify(direct - chain)`. `print(...)` displays results.

NumPy: nonlinear maps and column batches

Setup: `import numpy as np`. Create arrays with `np.array`, `np.zeros`, or `np.ones`; use shape `(m, n)` for a matrix. Use `@` for matrix products, `*` for entrywise products.

Column batches: `P` has shape `(2, N)`; each column is a point.

```
def F(P):
    x, y = P[0, :], P[1, :]
    return np.vstack([x + y**2, y])
```

`np.vstack` stacks rows; `F(P)` has shape `(2, N)`.

Sigmoid blocks and their Jacobians

Single input: shapes of `W1`, `W2` are `(h, n)`, `(m, h)`; shapes of `x`, `b1`, `b2` are `(n,)`, `(h,)`, `(m,)`.

```
s = W1 @ x + b1
u = 1 / (1 + np.exp(-s))
D = np.diag(u * (1 - u))
y = W2 @ u + b2
J_at_x = W2 @ D @ W1
```

`np.exp` acts entrywise; `np.diag` puts a vector on the diagonal. Here $D_{ii} = \sigma'(s_i) = u_i(1 - u_i)$; `J_at_x` has shape `(m, n)`. For column batches, biases need shapes `(h, 1)`, `(m, 1)` for broadcasting.

Softmax weights for column batches

Normalize each column: for score matrix `S = K @ P`,

```
E = np.exp(S - S.max(axis=0, keepdims=True))
Alpha = E / E.sum(axis=0, keepdims=True)
Q = V.T @ Alpha
```

`axis=0` reduces over rows; `keepdims=True` keeps shape `(1, N)` for broadcasting. Subtracting the maxima stabilizes the same softmax weights. With k keys, `Alpha` has shape `(k, N)` and columns summing to 1. For `V` of shape `(k, p)`, `Q` has shape `(p, N)`.

Gradient updates and diagnostics

Descent: for a positive learning rate `alpha`,

```
x = x0
for k in range(num_steps):
    x = x - alpha * grad_f(x)
```

Match `grad_f(x).shape` to `x.shape`. A positive `alpha` alone does not ensure convergence. For $\|X\mathbf{w} - \mathbf{y}\|^2$, set `r = X @ w - y` and `g = 2 * X.T @ r`; `r @ r` is the loss.

Matrix update: `G = np.outer(g, h)` forms $\mathbf{g}\mathbf{h}^T$, of shape `(m, d)` for vectors of lengths m, d. Use `W_new = W - alpha * G`. `np.round(value, 2)` rounds to two decimal places.